Assignment 3: Fixing a Slow Quicksort

AI Use Declaration: OpenAI ChatGPT/Codex was used to assist with Python implementation, test development, performance benchmarking, and explanation of algorithmic concepts. All generated code and experimental results will be reviewed and verified by the student before submission.

1. Introduction

A bank in Thimphu sorts mobile-payment amounts. Already ordered records and repeated Nu. 100 and Nu. 500 top-ups expose basic quicksort's worst case. This experiment compares a last-pivot implementation with a randomized, duplicate-aware version, measuring correctness, speed and practical limits.

2. Implementation

Basic: sorts.py uses Lomuto partitioning. A boundary separates amounts no greater than the last-element pivot from larger amounts. After scanning, the pivot moves to its final position, and both sides are recursively sorted. Balanced partitions give T(n) = 2T(n/2) + Theta(n) = Theta(n log n). For random distinct-key permutations, averaging pivot ranks also gives Theta(n log n). This average does not cover every repeated-key distribution. Extremely unbalanced partitions give Theta(n^2) time and Theta(n) stack space.

Improved: select a random pivot and partition into less-than, equal-to and greater-than regions. The equal region needs no further sorting; all-equal data takes Theta(n) time. Use insertion sort for partitions at most cutoff elements long. Recurse only on the smaller unequal region, then loop over the larger. Each recursive child is at most half its parent's size, giving O(log n) stack depth regardless of pivot quality. Expected time for fixed cutoff is O(n log n); worst-case time remains O(n^2). Both versions modify the supplied list and return None, without built-in sorting or array slices inside the algorithms.

3. Testing Results

The executed unittest suite passed all four test methods. It checks 500 seeded random lists of lengths 0-200 against sorted(), using basic quicksort and six improved cutoffs (1, 4, 8, 16, 32, 64): 3,500 random algorithm/cutoff checks. Additional cases cover empty, singleton, all-equal, ascending, descending, negative, repeated and mixed-sign inputs. Tests check alias-visible mutation, unchanged list identity, None returns and agreement between algorithms. The improved version also passed three 10,000-element ordered/equal cases; invalid cutoffs raise ValueError. Small basic test inputs avoid intentional stack failure.

Environment and reproduction: CPython 3.14.4, macOS 26.5, ARM64; default recursion limit 1,000. Run .venv/bin/python -m unittest -v, then MPLCONFIGDIR=/tmp/quicksort-mpl .venv/bin/python benchmark.py. Full metadata is in environment.txt. Timings are machine-specific wall-clock measurements, not theoretical operation counts.

4. Performance Analysis

Table 1. Median seconds, three trials where successful. B = basic, I = improved (cutoff 16). CRASH means actual RecursionError; it is not a zero timing.

n	Random B / I	Sorted B / I	Duplicates B / I
1,000	0.001196 / 0.001871	CRASH / 0.001917	0.007084 / 0.000513
2,000	0.003061 / 0.004207	CRASH / 0.004423	0.027461 / 0.000923
4,000	0.006525 / 0.009233	CRASH / 0.009672	0.106040 / 0.001870
8,000	0.013960 / 0.020446	CRASH / 0.021574	0.419052 / 0.003609
16,000	0.028462 / 0.044704	CRASH / 0.063812	CRASH / 0.007982
32,000	0.064063 / 0.096817	CRASH / 0.104413	CRASH / 0.014910
64,000	0.135071 / 0.206901	TIMEOUT / 0.209143	CRASH / 0.030824

Inputs are shared between algorithms, with fresh copies per run. Random values span 0-999,999; duplicate values span 0-9. Input seeds are 302 + n; pivot seeds are 302-304. Only sorting is timed with perf_counter(); copying, process startup and sorted() verification are excluded. Each case runs independently in fresh processes. A five-second worker timeout includes startup/verification; first crash or timeout ends that case. Successful medians require three completed trials. CSVs retain status, completed counts and trial times. Basic was faster on random data (0.135071 s versus 0.206901 s at n = 64,000). With 8,000 duplicate-heavy values, improved took 0.003609 s versus basic's 0.419052 s. There were 9 crashes, 1 timeouts and no skipped cases. Failures have blank CSV timing cells.

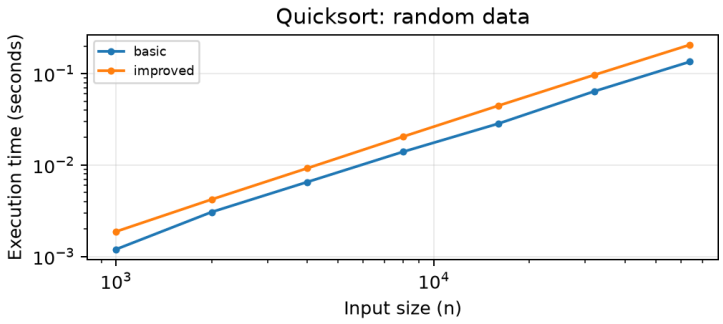


Figure 1. Random data; median seconds.

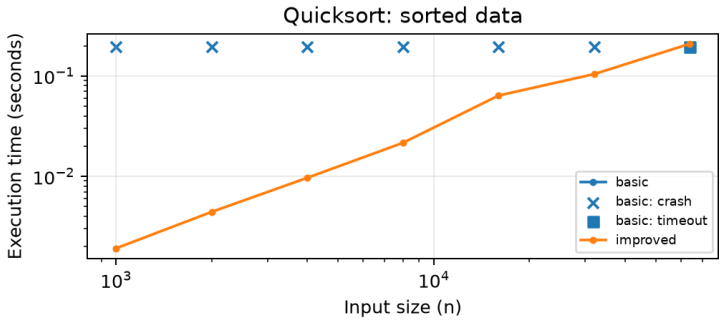


Figure 2. Ascending data; crosses = crashes, square = timeout (no time values).

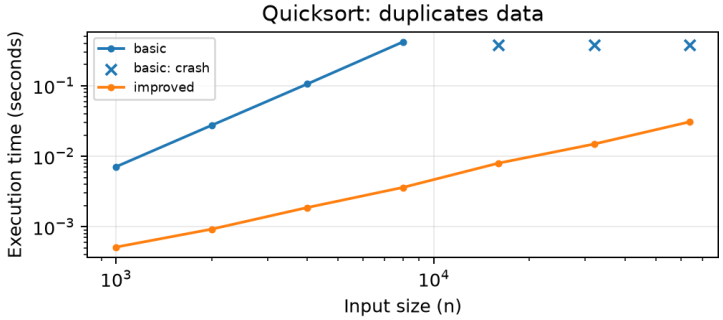


Figure 3. Duplicate-heavy data; crosses mark crashes, with no time value.

5. Worst-Case Recurrence Analysis

On ascending input, the last pivot is the maximum. All n - 1 preceding values pass Lomuto's <= test, leaving partitions of sizes n - 1 and zero. Scanning the partition costs Theta(n), even though the data is already ordered. Thus T(n) = T(n - 1) + Theta(n), with a constant-size base case. Expanding gives Theta(n + (n - 1) + ... + 1) = Theta(n(n + 1)/2) = Theta(n^2). There are Theta(n) nested calls, so Python's finite recursion limit causes RecursionError. With all equal amounts, the <= test again puts every other element on one side; randomizing the pivot alone cannot fix this. Three-way partitioning removes all copies of a pivot value together, making repeated bank top-ups inexpensive.

6. Cutoff Optimization

Table 2. Three-trial medians on the same 100,000 random integers.

Cutoff	1	4	8	16	32	64
Median seconds	0.375225	0.356902	0.344982	0.346430	0.346177	0.376319

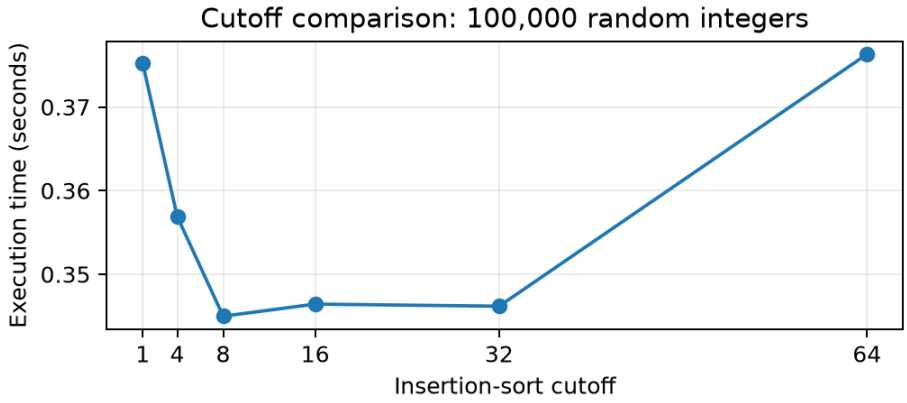


Figure 4. Insertion-sort cutoff experiment.

The measured best cutoff is **8**, at **0.344982 seconds**. Small insertion sorts avoid recursive partitioning overhead and have simple loops. Their quadratic cost is bounded by the small partition size: across leaves the cost is O(nk) for cutoff k. Very large cutoffs increase shifting work. This small experiment does not establish statistical significance or a universal best cutoff.

7. Comparison with Python's sorted()

On the same seeded 100,000-element input, a separate three-trial comparison measured improved quicksort (cutoff 8) at **0.373789 s** and sorted() at **0.017612 s**. The ratio improved / sorted is **21.22**, so sorted() was 21.22 times faster. sorted() includes allocating its result; both exclude input copying. CPython's adaptive Timsort exploits existing runs and executes optimized C code; our swaps, comparisons, loops and calls execute through Python. This experiment cannot numerically separate algorithmic effects from language overhead. sorted() produces a new list and is stable; our quicksorts are in place and unstable.

8. Critical Reflection

Question 1: Why did the basic algorithm fail? On ascending transaction amounts, the final pivot is always the maximum. Lomuto therefore leaves an almost unchanged subproblem and creates linear recursion depth. In these experiments, 9 basic input-size cases raised RecursionError, including 6 ascending cases; the largest ascending case had status timeout. The average-case analysis assumes random relative input order, or an equivalent source of pivot randomness; the bank's already ordered records break that assumption. Repeated top-ups also concentrate equal keys on one side in the basic version. Random pivots help with order, while three-way partitioning directly addresses duplicates.

Question 2: Why use insertion sort? A quadratic algorithm can still be useful on a short partition because it avoids further partitioning, pivot selection and function calls. For fixed cutoff k, insertion work across the small partitions is O(nk). The measured fastest cutoff was 8, with a median of 0.344982 seconds for 100,000 random integers. This is a result for this implementation and machine, not a universal optimum. A bank should remeasure with representative transaction data before selecting its cutoff.

Question 4: How was AI assistance checked? ChatGPT/Codex assisted with the implementation, tests, benchmark and explanation. Automated checks compared both algorithms with Python's sorted() on 500 reproducible random inputs and explicit edge cases. The benchmark independently verified every successful output and preserved crashes as missing times. This checks correctness and measured behavior, but cannot replace understanding the partition invariants or reviewing experimental design. Student review required: personally rerun the commands, inspect the code and CSV evidence, and confirm this declaration accurately describes your use of AI before submission.

9. Conclusion

Random pivot selection reduces sensitivity to input order, three-way partitioning handles repeated amounts, and smaller-side recursion bounds stack growth. The basic version's observed failures explain why a method that works on random payments may fail on routine bank data. The measured cutoff 8 performed best in the cutoff sweep; Python's sorted() was faster in the direct comparison. The evidence supports the improved implementation for this assignment, while a bank using Python should also consider the built-in stable sorter.

Student action before submission: review the AI declaration and reflection, rerun tests and benchmarks if required by the course, add your name/student number, and submit sorts.py, test_sorts.py and report.pdf. The scripts, CSVs and four PNGs remain supporting evidence. Submission timing and final approval belong to the student.